GESTIÓN DE CONFIGURACIÓN Y MANTENIMIENTO DE SOFTWARE · UNIDAD 4

Recorrido resuelto

Los formatos F-22 a F-26 con el ejercicio de refactorización del curso y con Flask · Maestría en Ingeniería de Software, Universidad de La Sabana

Este recorrido usa dos casos reales. El primero es **el ejercicio de la carpeta `ejercicio/`**: un módulo heredado de tarifas de envío, refactorizado en cuatro pasos. Cada paso es un commit del repositorio del curso y cada uno tiene su ejecución de la CI en verde. El segundo es **Flask**: su reestructuración de la versión 3.0.0, medida con la misma herramienta. Datos del 1 de noviembre de 2026.

Primera parte · El ejercicio

F-22 · Oportunidades de mejora

#	Mejora	De dónde sale	Valor	Costo	Se hace
1	Que cambiar una tarifa sea cambiar un solo lugar	Deuda: el cálculo está copiado tres veces, una por zona	Alto	Bajo	Sí
2	Separar el formato de la factura del cálculo	Deuda: el formato de pesos está repetido	Medio	Bajo	Sí
3	Corregir el redondeo de los .5	Hallazgo de las pruebas de caracterización	Medio	Bajo	No: cambia el comportamiento. Queda como deuda, para un commit propio y una decisión del negocio

F-23 · Estrategia

Refactorizar en el lugar. Los problemas están localizados en un solo archivo, el comportamiento se puede fijar con pruebas y el negocio necesita lo mismo de siempre. Reemplazar el módulo sería más riesgoso y no aportaría nada.

F-24 · Plan de refactorización

Pruebas de caracterización: 24 casos en 7 funciones de prueba. Cubren tarifas por zona y peso, envío express, descuentos del cliente frecuente, redondeo, zona desconocida y factura. Al escribirlas apareció una rareza: un envío urbano de 1,3 kg cuesta 8.450 y el módulo lo cobra 8.400, no 8.500, porque `round()` de Python lleva los .5 al número par. La prueba fija el comportamiento real y la rareza queda registrada.

#	Paso	Commit	Ejecución de la CI	¿Cambia el comportamiento?
—	Código heredado y pruebas de caracterización	`5e6bf7c`	41317571750 · verde	—
1	Extrae el cálculo por peso que se repetía en las tres zonas	`06216a6`	41317648643 · verde	No: 24 de 24
2	Reemplaza los condicionales por zona con una tabla de tarifas	`8f1e225`	41317704273 · verde	No: 24 de 24
3	Extrae el descuento por cliente y el redondeo a centenas	`d51f577`	41317765998 · verde	No: 24 de 24
4	Elimina la duplicación del formato de la factura	`3f2ee8f`	41317840174 · verde	No: 24 de 24

La ejecución 41317571750 corresponde al commit siguiente al del código heredado, que solo sacó del repositorio unos archivos compilados de Python; el código es el mismo. La CI escribe el impacto de cada paso en el resumen de su ejecución.

F-25 y F-26 · Medición y evaluación del impacto

Objetivo declarado: que cambiar una tarifa sea cambiar un solo lugar, y que el código sea más fácil de leer y de probar. **Métricas:** complejidad máxima, anidamiento, líneas duplicadas y tamaño de las funciones.

```
python herramientas/impacto.py . --antes 5e6bf7c --despues 3f2ee8f --ruta ejercicio/envios --
pruebas ejercicio/tests
```

Métrica	Heredado	Paso 1	Paso 2	Paso 3	Paso 4
Líneas de código	51	39	35	35	39
Funciones	2	3	3	5	7
Complejidad total	19	16	12	14	15
Complejidad máxima	15	9	5	4	3
Complejidad promedio	9,5	5,3	4,0	2,8	2,1
Anidamiento máximo	5	4	2	1	1
Líneas duplicadas	8	6	0	0	0
Funciones de más de 25 líneas	1	0	0	0	0

Qué mejoró	La función principal pasó de 41 líneas y complejidad 15 a 8 líneas y complejidad 3. No queda duplicación. Cambiar una tarifa es cambiar una fila de `TARIFAS`.
Qué no mejoró, y por qué	La complejidad total subió en los pasos 3 y 4 (de 12 a 15) y las líneas de código en el paso 4 (de 35 a 39): hay más funciones, cada una simple y con nombre. Por eso se miran varias métricas y se lee el objetivo, no una cifra.
El comportamiento	Se conservó: las 24 pruebas pasan en los cinco estados, en la CI.
Deuda que queda	El redondeo de los .5. Corregirlo es un cambio de comportamiento: se haría en un commit propio, con las pruebas actualizadas a propósito y una entrada en el registro de cambios.

Segunda parte · Flask 3.0

La versión 3.0.0 reorganizó las clases principales de Flask sobre bases sin entrada y salida (*Sans-IO*, PR 5127) y retiró código obsoleto. Medida con la misma herramienta sobre `src/flask`:

```
python herramientas/impacto.py flask --antes 2.3.3 --despues 3.0.0 --ruta src/flask --pruebas
tests
```

Métrica	2.3.3	3.0.0
Archivos	22	24
Líneas de código	6.402	6.460
Funciones	357	354
Complejidad máxima	30	23
Funciones con complejidad mayor que 10	10	10
Líneas duplicadas	477	775
Funciones de prueba	368	369

Lectura. La función más compleja, `register` de `blueprints.py`, bajó de 30 a 23. La duplicación subió porque `app.py` y el nuevo `sansio/app.py` comparten definiciones copiadas, como `_make_timedelta` y las variables de tipos: 90 de los 134 bloques repetidos nuevos están entre esos dos archivos. El objetivo de la reestructuración era separar el núcleo de la entrada y salida, no reducir estas métricas. Sin el objetivo declarado, la tabla diría que el código empeoró.

El detalle por función está en `ejemplo_flask/impacto_funciones_2.3.3_a_3.0.0.csv`.

Lo que este recorrido enseña para su entrega

1. **Escriban las pruebas de caracterización antes de tocar el código.** En el ejercicio, una de ellas encontró una rareza que nadie conocía.
2. **Un commit por paso, con la CI en verde.** Si algo sale mal, se revierte un paso, no la semana.
3. **Declaren el objetivo antes de medir.** Flask 3.0 muestra que las métricas genéricas pueden empeorar mientras el diseño mejora en lo que se buscaba.
4. **Separen refactorizar de cambiar el comportamiento.** La rareza del redondeo se registró, no se corrigió de paso.

Referencias

Feathers, M. (2004). *Working effectively with legacy code*. Prentice Hall.

Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2.ª ed.). Addison-Wesley.

Pallets. (2026). *Flask* [Repositorio de código fuente]. GitHub. https://github.com/pallets/flask